Задание: по заданным описаниям таблиц и вставок данных определить используемый движок, заполнить пропуски в `<ENGINE>` и запустить код; сравнить полученный вывод с результатом из условия.

tbl1 — VersionedCollapsingMergeTree

Листинг скрипта

```
-- tbl1: VersionedCollapsingMergeTree
USE homework7;

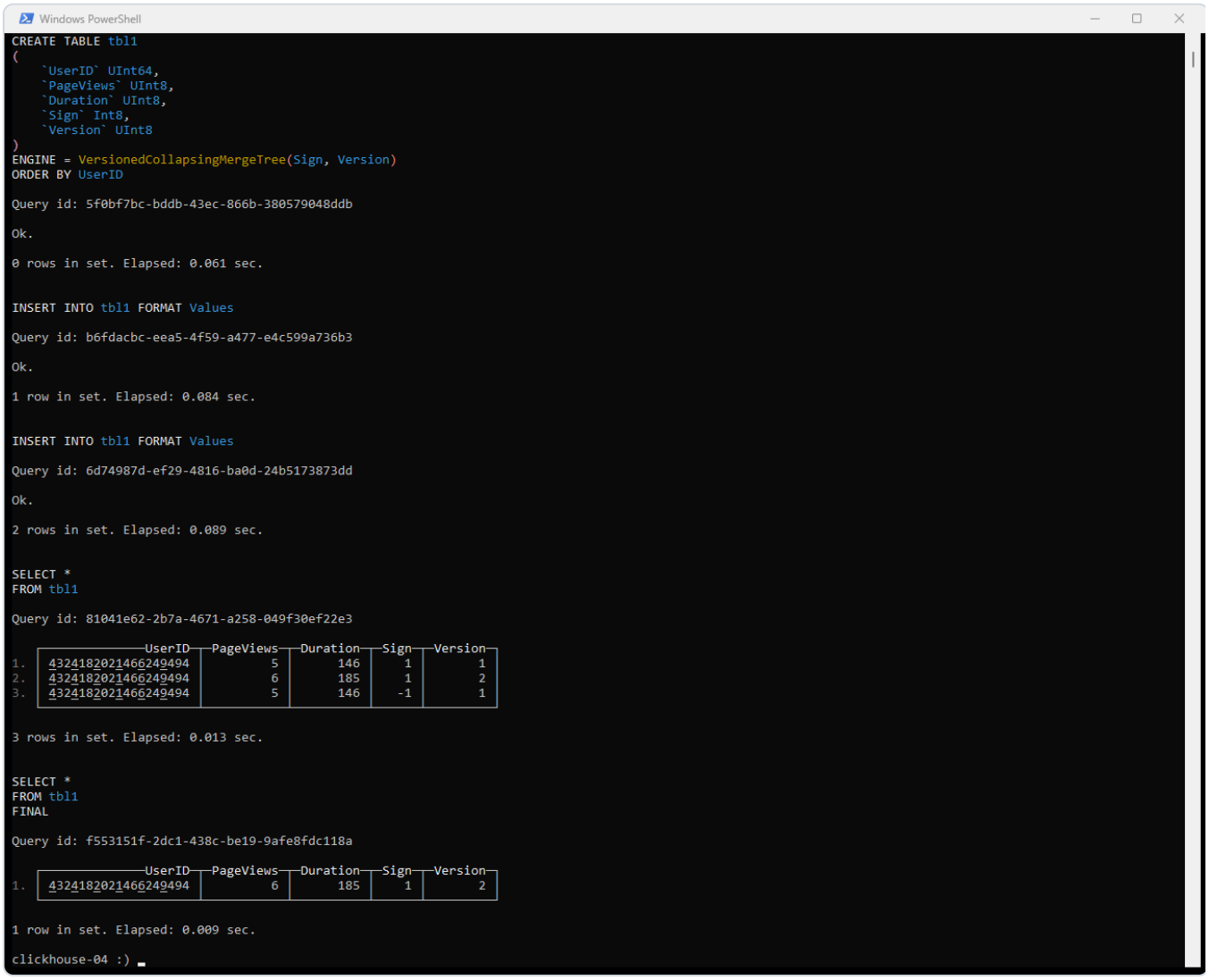
DROP TABLE IF EXISTS tbl1;

CREATE TABLE tbl1
(
    UserID    UInt64,
    PageViews UInt8,
    Duration  UInt8,
    Sign      Int8,
    Version   UInt8
)
ENGINE = VersionedCollapsingMergeTree(Sign, Version)
ORDER BY UserID;

INSERT INTO tbl1 VALUES (4324182021466249494, 5, 146, -1, 1);
INSERT INTO tbl1 VALUES
    (4324182021466249494, 5, 146, 1, 1),
    (4324182021466249494, 6, 185, 1, 2);

SELECT * FROM tbl1;
SELECT * FROM tbl1 FINAL;
```

Скриншот результата



Обоснование

Sign работает с положительными и отрицательными значения для свертки, плюс на минус а Version дает версию строки. VersionedCollapsingMergeTree использует обе колонки: при FINAL схлопывается пара -1/+1 с одинаковой версией, а строка с Version = 2 остаётся как актуальное состояние.

tbl2 — SummingMergeTree

Листинг скрипта

```
-- tbl2: SummingMergeTree
USE homework7;
```

```sql
DROP TABLE IF EXISTS tbl2;

CREATE TABLE tbl2
(
    key   UInt32,
    value UInt32
)
ENGINE = SummingMergeTree
ORDER BY key;

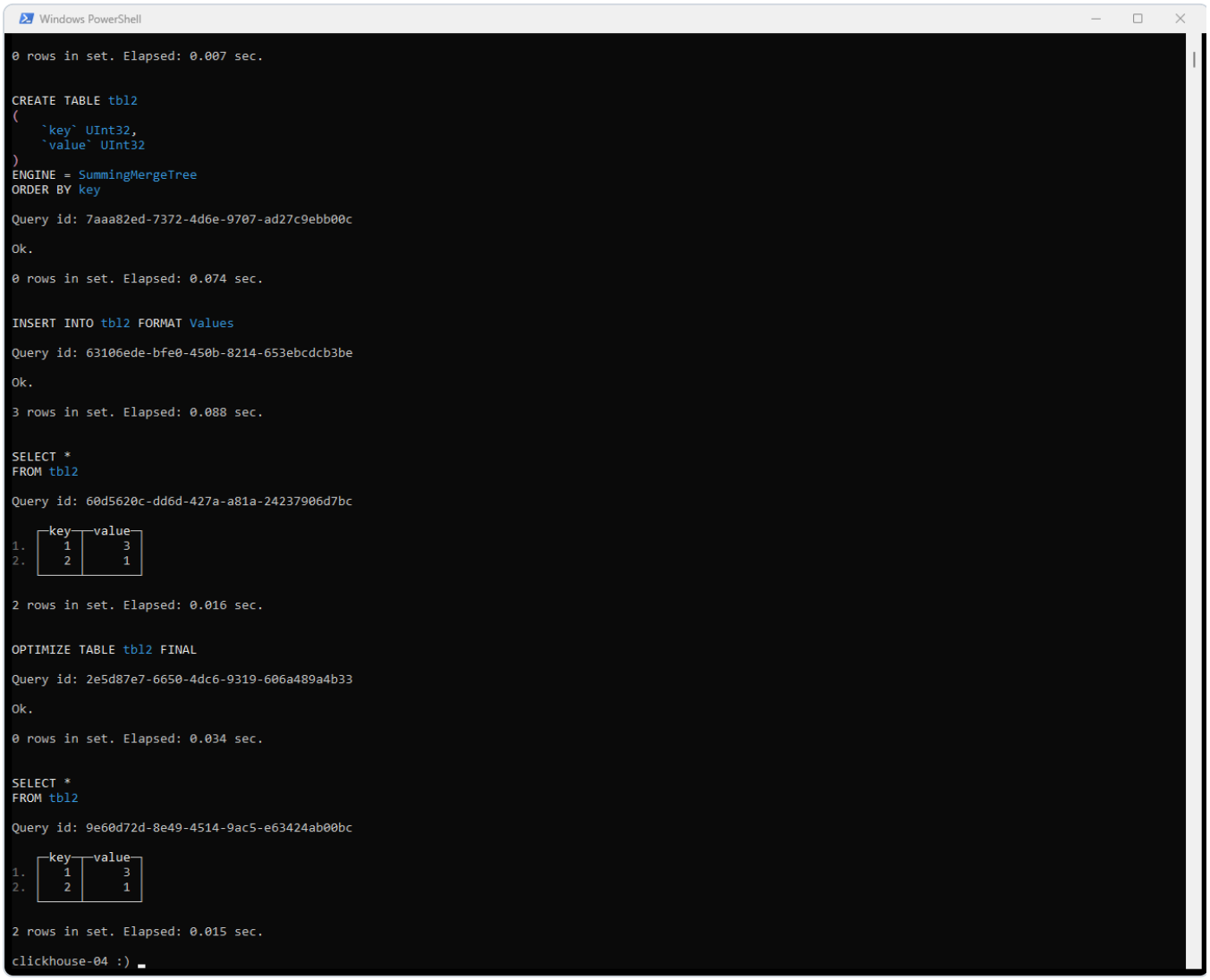
INSERT INTO tbl2 VALUES (1, 1), (1, 2), (2, 1);

SELECT * FROM tbl2;

OPTIMIZE TABLE tbl2 FINAL;

SELECT * FROM tbl2;
```

Скриншот результата



Обоснование

key - ключ, valu - значение. SummingMergeTree при merge суммирует value для строк с одинаковым key. В этом запуске фоновый merge уже успела выполниться.

tbl3 — ReplacingMergeTree

Листинг скрипта

```
-- tbl3: ReplacingMergeTree
USE homework7;

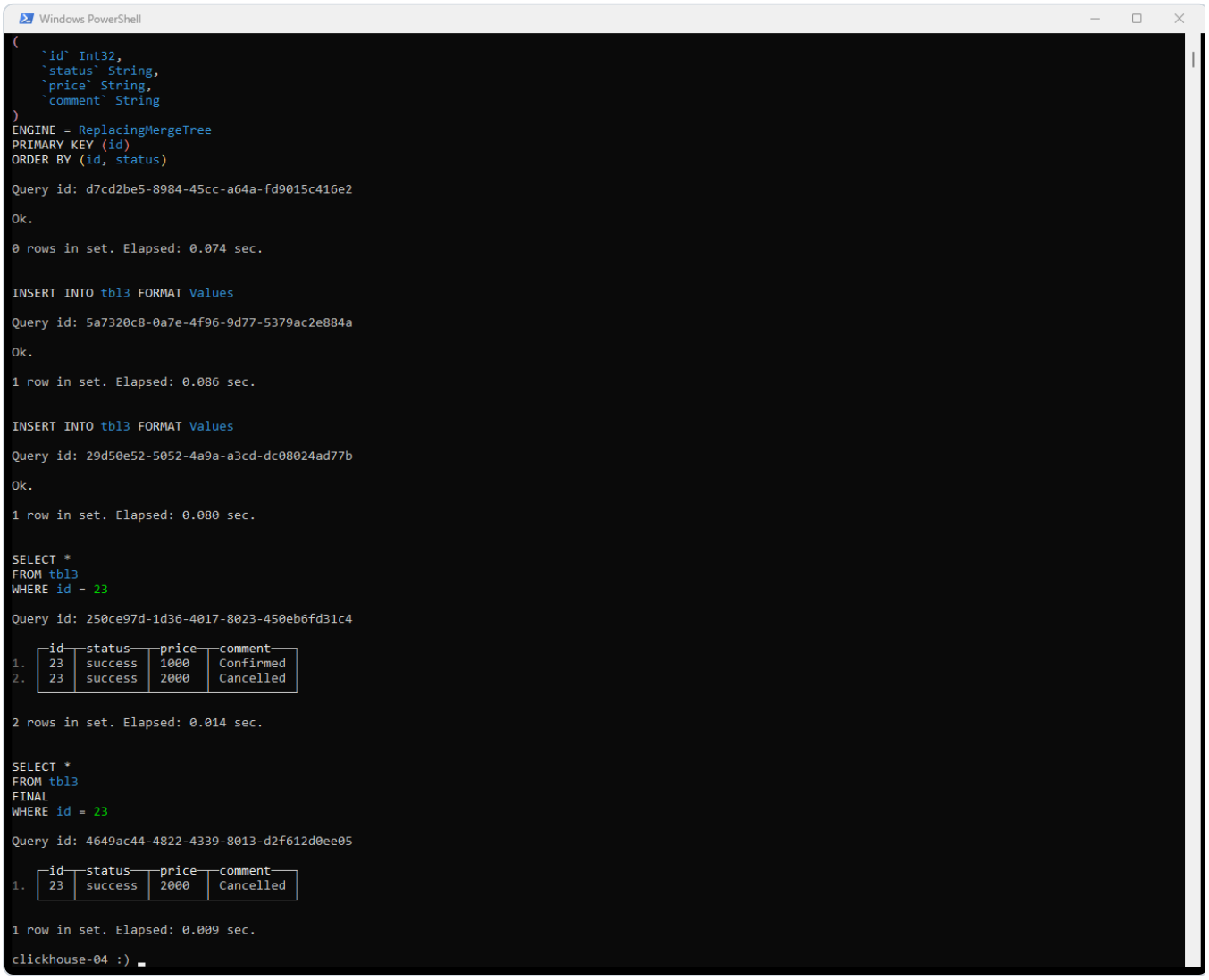
DROP TABLE IF EXISTS tbl3;

CREATE TABLE tbl3
(
    `id`      Int32,
    `status`  String,
    `price`   String,
    `comment` String
)
ENGINE = ReplacingMergeTree
PRIMARY KEY (id)
ORDER BY (id, status);

INSERT INTO tbl3 VALUES (23, 'success', '1000', 'Confirmed');
INSERT INTO tbl3 VALUES (23, 'success', '2000', 'Cancelled');

SELECT * FROM tbl3 WHERE id = 23;
SELECT * FROM tbl3 FINAL WHERE id = 23;
```

Скриншот результата



Обоснование

Две строки имеют одинаковый primary key id и одинаковый ключ сортировки id, status, но разные значения остальных полей. Нужно оставить одну актуальную версию — для этого подходит `ReplacingMergeTree`. Без `FINAL` видны обе строки, с `FINAL` остаётся последняя вставленная (`2000`, `Cancelled`). Схлопывает по порядку вставки.

tbl4 — MergeTree

Листинг скрипта

```
-- tbl4: MergeTree
USE homework7;
```

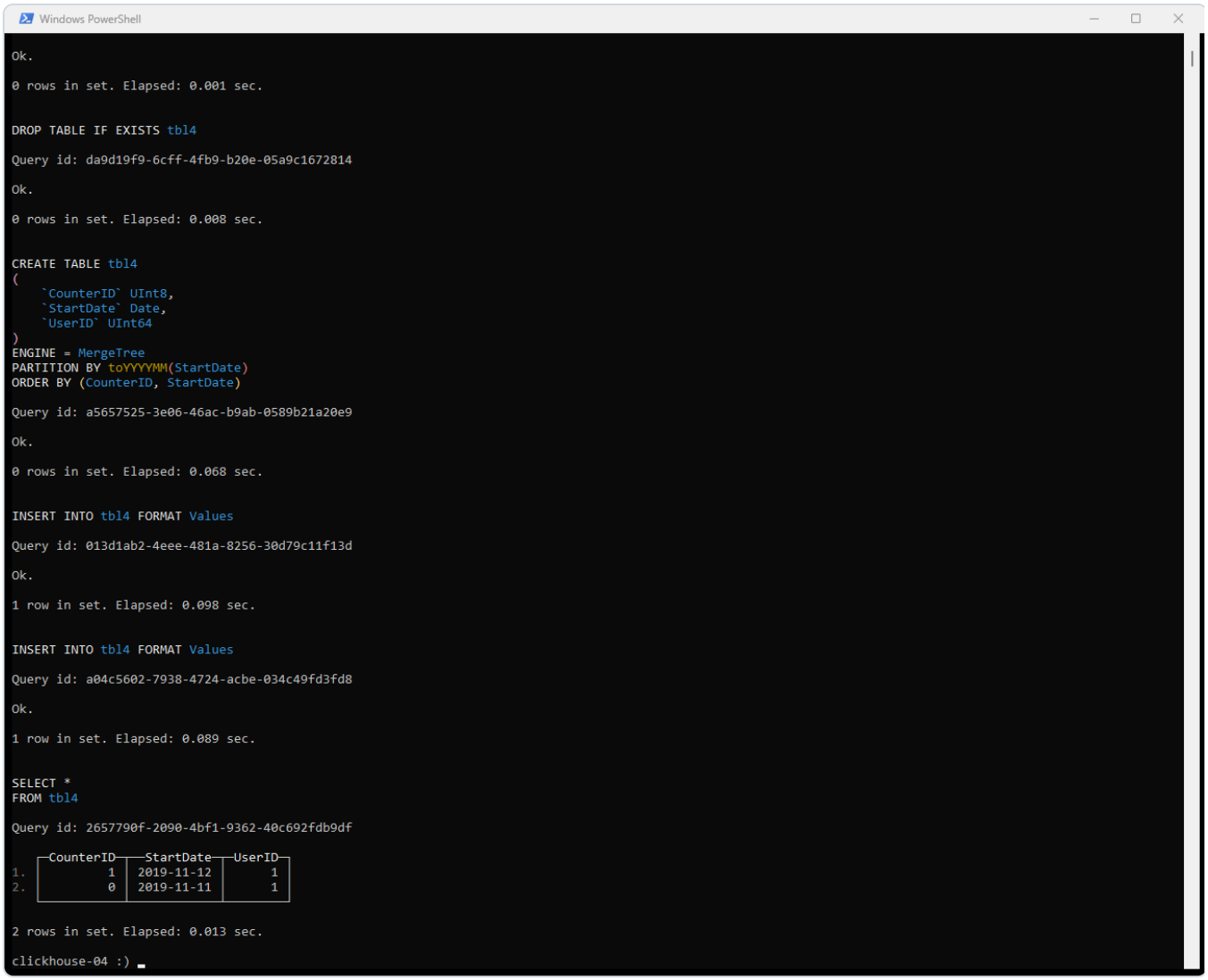
```sql
DROP TABLE IF EXISTS tbl4;

CREATE TABLE tbl4
(
    CounterID UInt8,
    StartDate Date,
    UserID    UInt64
)
ENGINE = MergeTree
PARTITION BY toYYYYMM(StartDate)
ORDER BY (CounterID, StartDate);

INSERT INTO tbl4 VALUES (0, '2019-11-11', 1);
INSERT INTO tbl4 VALUES (1, '2019-11-12', 1);

SELECT * FROM tbl4;
```

Скриншот результата



Обоснование

Это обычная таблица хранения, нет дубликатов, агрегатных состояний или схлопывания. MergeTree подходит как базовая таблица с партиционированием по месяцам.

5. tbl5 — AggregatingMergeTree

Листинг скрипта

```
-- tbl5: AggregatingMergeTree
USE homework7;

DROP TABLE IF EXISTS tbl5;

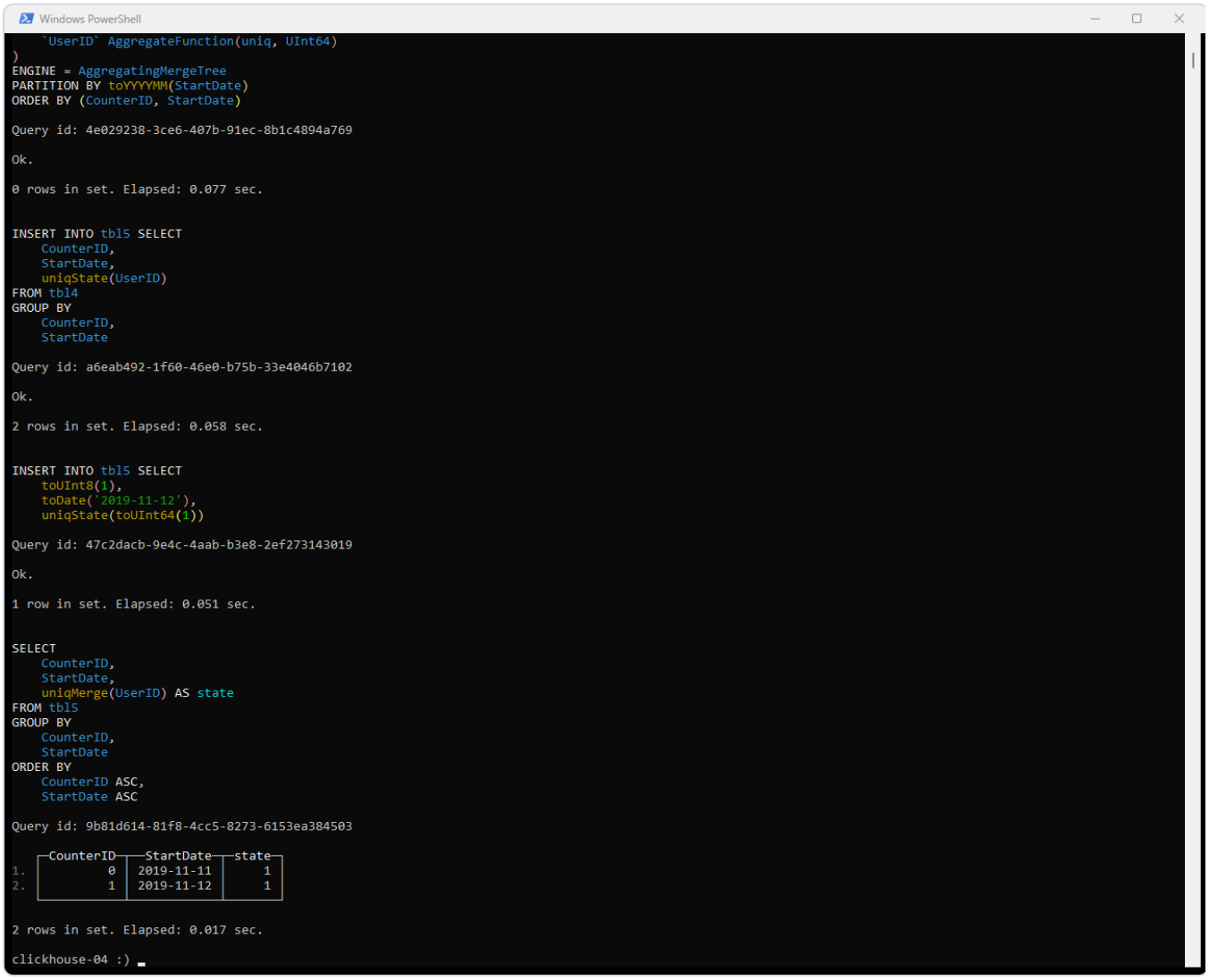
CREATE TABLE tbl5
(
    CounterID UInt8,
    StartDate Date,
    UserID    AggregateFunction(uniq, UInt64)
)
ENGINE = AggregatingMergeTree
PARTITION BY toYYYYMM(StartDate)
ORDER BY (CounterID, StartDate);

INSERT INTO tbl5
SELECT CounterID, StartDate, uniqState(UserID)
FROM tbl4
GROUP BY CounterID, StartDate;

INSERT INTO tbl5
SELECT toUInt8(1), toDate('2019-11-12'), uniqState(toUInt64(1));

SELECT CounterID, StartDate, uniqMerge(UserID) AS state
FROM tbl5
GROUP BY CounterID, StartDate
ORDER BY CounterID, StartDate;
```

Скриншот результата



Обоснование

Колонка `UserID` имеет тип `AggregateFunction(uniq, UInt64)`, то есть хранит не число, а состояние агрегата. `AggregatingMergeTree` предназначен именно для таких состояний: вставка через `uniqState`, чтение через `uniqMerge`, партиционирование по месяцам и сортировка по `(CounterID, StartDate)`.

tbl6 — CollapsingMergeTree

Листинг скрипта

```
-- tbl6: CollapsingMergeTree
USE homework7;
```

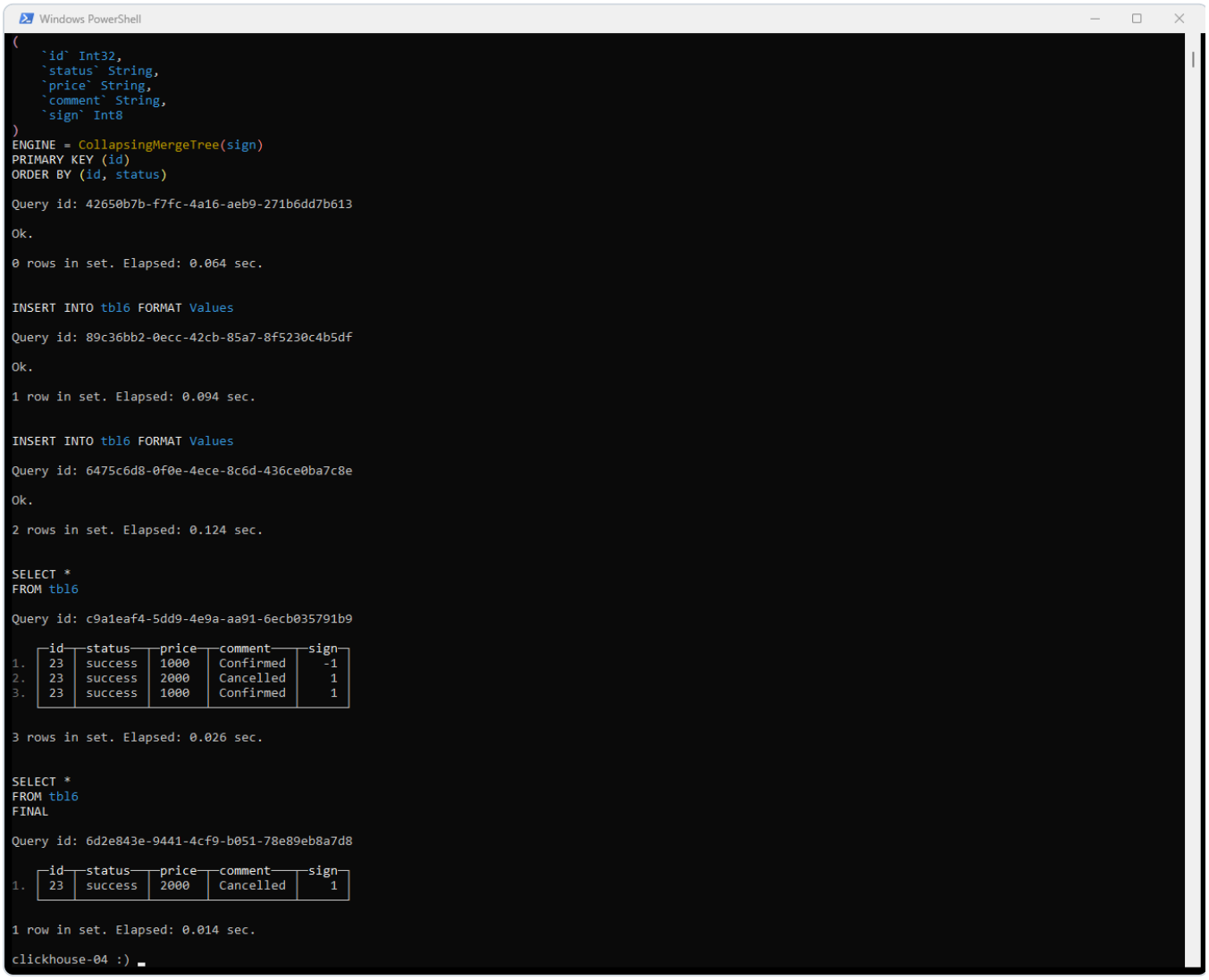
```
DROP TABLE IF EXISTS tbl6;

CREATE TABLE tbl6
(
    `id`      Int32,
    `status`  String,
    `price`   String,
    `comment` String,
    `sign`    Int8
)
ENGINE = CollapsingMergeTree(sign)
PRIMARY KEY (id)
ORDER BY (id, status);

INSERT INTO tbl6 VALUES (23, 'success', '1000', 'Confirmed', 1);
INSERT INTO tbl6 VALUES
    (23, 'success', '1000', 'Confirmed', -1),
    (23, 'success', '2000', 'Cancelled', 1);

SELECT * FROM tbl6;
SELECT * FROM tbl6 FINAL;
```

Скриншот результата



Обоснование

Есть колонка sign, но нет колонки версии. Для отмены предыдущей строки используется -1, для добавления 1. CollapsingMergeTree(sign) при FINAL схлопывает пару 1/-1 с одинаковым ключом сортировки.

7. Проблемы и решения

1. **Вставка в AggregateFunction.** Исходный запрос `INSERT INTO tbl5 VALUES (1, '2019-11-12', 1);` завершается ошибкой:

``` `text Code: 53. DB::Exception: Cannot convert UInt64 to AggregateFunction(uniq, UInt64): While executing WaitForAsyncInsert. (TYPE_MISMATCH)` ```
```

`AggregateFunction` хранит сериализованное состояние, а не обычное значение. Решение — вставлять через `uniqState` и явно приводить типы:

```sql
INSERT INTO tbl5 SELECT toUInt8(1), toDate('2019-11-12'), uniqState(toUInt64(1));
```

Литерал `1` имеет тип `UInt8`, поэтому для колонки `UInt64` требуется `toUInt64`.

2. **`FINAL` и несмерженные пары.** В `CollapsingMergeTree` / `VersionedCollapsingMergeTree` без `FINAL` видны все физические строки. `FINAL` схлопывает только корректные пары `+1/-1`; несвёрнутая строка остаётся. Для `VersionedCollapsingMergeTree` версии должны совпадать, иначе пара не схлопнется.